

CURSOR AI IDE: BUILDING PRODUCTION APPLICATIONS

O'Reilly Live Training — 6 Hours · 10 Sections · Instructor: Lucas Soares

Arc: HTML toy app → Simple quiz → AI-powered quiz app → Backend/Auth/Payments → Deployment

TECH STACK — DEMO PROJECT (AI QUIZ APP)

Next.js 15+ App Router

TypeScript

Tailwind CSS

shadcn/ui

Supabase (PostgreSQL)

Drizzle ORM

Clerk Auth

Stripe

OpenAI / DeepSeek API

Vitest

Playwright

Vercel

01 CURSOR INTRO (~45 MIN) — FOUR INTERACTION MODES

Four Cursor Modes

- **Chat (Cmd+L):** Explore, explain, ask questions
- **Agent:** Multi-file autonomous changes
- **Inline Edit (Cmd+K):** Quick targeted edits
- **Tab Autocomplete:** Predict and accept completions

Key Principles

- Cursor is **AI-first**, not AI-bolted-on
- **"Try 5 times"** rule — rephrase & retry if a prompt fails
- Git commits are critical — Cursor is **NOT** a checkpoint system
- Quick edit → Inline; multi-file → Agent; explore → Chat

Demo Prompt — Toy HTML App

Create a single-file HTML page with embedded CSS and JS: centered card titled "My First Cursor App", counter with + and - buttons, color theme picker (3-4 presets), clean modern styling, no external deps.

02 EXPLORE-PLAN-BUILD FRAMEWORK (~45 MIN)

1

EXPLORE

Understand codebase with @ symbols before writing a line of code

2

PLAN

Design approach, identify files, think through data flow

3

BUILD

Implement with clear curated context — not everything

@ Symbol Reference

- **@file** — Reference specific file (precision)
- **@folder** — Reference a directory (summary)
- **@codebase** — Semantic search across project
- **@docs** — Query documentation
- **@web** — Search the web for current info

Two-Stage Workflow

- **Stage 1:** Use ChatGPT / Claude for planning & design
- **Stage 2:** Use Cursor for implementation
- Context file = project "north star" (saves repeating yourself)
- More context ≠ better results — curate what AI sees

03 PROJECT RULES (~20 MIN) — HIERARCHY OF LEVERAGE

Hierarchy of Leverage: Rules > Context > Prompts > Model Capability

Four Rule Application Modes

- **Always:** Applied to every request
- **Auto Attached:** Triggers on file pattern (e.g., `*.tsx`)
- **Agent Requested:** AI decides when to apply
- **Manual:** Only when explicitly referenced

Rules Best Practices

- Rules = onboarding docs for your AI (encode team standards)
- Start **without** rules, add only when mistakes repeat
- `AGENTS.md` (Cursor) and `CLAUDE.md` (Claude Code) = project context
- Don't over-engineer — rules add overhead to every request
- Location: `.cursor/rules/*.mdc`

04 MCP & TOOLING (~30 MIN) — "USB FOR AI"

MCP (Model Context Protocol)

- Open protocol by Anthropic — standardizes AI ↔ tool connections
- One interface, many devices (like USB)
- Key integrations: **Playwright, GitHub, GitMCP, databases**
- **GitMCP trick:** Replace `github.com` → `gitmcp.io`
- Playwright MCP: AI sees & interacts with your running app

Playwright MCP Config

```
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp"]
    }
  }
}
```

CLI vs MCP Comparison

Tool	Interface	Best For	Git	Visual Context
Cursor (IDE + MCP)	Visual IDE	UI/visual work, live demos	Basic	Playwright MCP
Claude Code (CLI)	Terminal	Infrastructure, scripts, SSH	Deep (auto-commits)	Screenshot tools
Aider (CLI)	Terminal	Quick targeted code changes	Deep	No

05 AI QUIZ APP BUILD (~60 MIN) — FULL-STACK SAAS

App Architecture & Design Decisions

- Landing → Dashboard → Create Quiz → Take Quiz → Results
- AI gen: text/PDF → Claude API (two-stage) → structured JSON
- Quiz state managed in React hook (no Redux/Zustand)
- Questions stored as JSONB in Postgres (no separate table)
- AI generation server-side only — API key never touches client
- Exports (CSV, Markdown, Anki) run client-side

Implementation Phases

- **Phase 0:** Project setup (Next.js + Shadcn + deps)
- **Phase 1:** Auth & user sync (Clerk → Supabase)
- **Phase 2:** Quiz data layer (Zod + CRUD + RLS)
- **Phase 3:** Quiz creation page (upload + AI generate)
- **Phase 4:** Quiz player (state machine + animations)
- **Phase 5:** Results + export (CSV, Markdown, Anki)
- **Phase 6:** Dashboard (library + performance stats)
- **Phase 7:** Stripe payments + plan gating
- **Phase 8:** Landing page + polish + SEO

Database Schema (key tables)

- `users` — id, email, stripe_customer_id, subscription_status
- `quizzes` — id, user_id, title, questions JSONB, is_public
- `quiz_attempts` — id, quiz_id, user_id, score, results JSONB

Quiz JSON Format

```
// Multiple choice
{ "type": "multiple-choice",
  "question": "...", "options":
  ["A", "B", "C", "D"],
  "correctAnswer": 0, "explanation": "..." }

// Open-ended
{ "type": "open-ended", "question": "...",
  "acceptedAnswers": ["answer"], "explanation":
  "..." }
```

06 TESTING (~20 MIN) — TDD WITH AI

TDD Workflow with Cursor

1. Write the **failing test FIRST** (you define behavior)
 2. Let Cursor implement the code to pass the test
 3. Refactor with confidence
- AI generates edge cases you'd likely miss
 - Playwright MCP: AI runs E2E tests & sees results visually

Testing Pyramid

- **Unit (Vitest):** Pure functions, utilities, validators
- **Component (RTL):** React components in isolation
- **Integration:** API routes, database queries
- **E2E (Playwright):** Full user flows in real browser

Many unit tests → fewer E2E tests. Write test → AI implements.

07 DIAGRAMS & DESIGN (~30 MIN) — SKETCH → CODE

Mermaid Diagram Types

- **Architecture diagrams:** System overview
- **User flow diagrams:** Sequence / flow charts
- **Database ERDs:** Entity relationships
- Text-based → perfect for version control & AI generation
- Use mermaid.live for preview without extension

Sketch → Code Workflow

1. Take photo / screenshot of design (hand-drawn or Figma)
 2. Paste in Cursor chat
 3. Get implementation code
 4. Iterate: "more spacious", "match this color scheme"
- AI iterates on designs from natural language descriptions

08 BACKEND, AUTH & PAYMENTS (~45 MIN)

Supabase + Drizzle ORM

- Drizzle = type-safe queries (errors at compile time)
- Row Level Security (RLS) isolates data per user
- Use connection pooling in production (not direct)
- Enable RLS on **all** tables before going live

Clerk Auth

- Middleware protects routes automatically (one config file)
- `ClerkProvider` wraps root layout
- `<UserButton />` renders user avatar + menu
- Webhook syncs Clerk users → Supabase `users` table
- Free tier: 10,000 monthly active users

Stripe Payments

- Test mode is identical to production (safe to demo live)
- Flow: Checkout → webhook → update `subscription_status`
- Gate AI generation: free = 3/month, pro = unlimited
- Monthly reset of `ai_generations_used` counter

Next.js Server Actions

- No separate API layer needed for mutations
- Type-safe from client to server
- Use `"use server"` directive at top of file or function
- McKay's template has most pre-wired — connect the dots

Required Environment Variables

Service	Variable(s)
Clerk	<code>NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY</code> · <code>CLERK_SECRET_KEY</code> · <code>CLERK_WEBHOOK_SECRET</code>
Supabase	<code>NEXT_PUBLIC_SUPABASE_URL</code> · <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code> · <code>SUPABASE_SERVICE_ROLE_KEY</code>
Stripe	<code>STRIPE_SECRET_KEY</code> · <code>NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY</code> · <code>STRIPE_WEBHOOK_SECRET</code>
AI + App	<code>ANTHROPIC_API_KEY</code> · <code>NEXT_PUBLIC_APP_URL</code>

09 DEPLOYMENT (~30 MIN) — VERCEL + PRODUCTION READINESS

Vercel Deployment Steps

1. Connect GitHub repo to Vercel (zero-config for Next.js)
2. Set all environment variables in Vercel dashboard
3. Configure webhook URLs: Clerk + Stripe → production domain
4. Deploy — automatic on push to `main`
 - **Env vars = #1 deployment issue** — double-check all

Free Tier Cost Breakdown

- **Vercel:** Free hobby — 100 GB bandwidth
- **Supabase:** Free — 500 MB DB, 2 GB storage
- **Clerk:** Free — 10,000 MAU
- **Stripe:** No monthly fee (transaction % only)
- **Anthropic API:** ~\$0.003 per quiz generated

Production Checklist

- ☐ Enable RLS on all Supabase tables
- ☐ Use connection pooling (not direct connection)
- ☐ Error boundaries on all pages
- ☐ All env vars set in production dashboard
- ☐ Stripe webhooks configured with production URL
- ☐ Set up monitoring / logging

10 TAKEAWAYS

- 1. Cursor is a **development multiplier**, not a replacement for thinking — you make the architectural decisions
- 2. **Explore** → **Plan** → **Build** — never skip the first two steps
- 3. **Context quality > prompt cleverness** — curate what the AI sees
- 4. **Project rules** encode team standards — add incrementally, not all at once
- 5. **MCP extends Cursor's reach** — Playwright, GitHub, databases, docs
- 6. **Two-stage workflow works** — plan with Claude/ChatGPT, build with Cursor
- 7. **TDD with AI** — write the test (define behavior), let AI implement
- 8. **Full-stack is achievable** — Supabase + Clerk + Stripe + Vercel = production app
- 9. **Git discipline is non-negotiable** — commit early, commit often
- 10. **Try 5 times before giving up** — AI prompting is a learnable skill

Core Resources

- **Cursor:** cursor.com · docs.cursor.com
- **MCP Registry:** github.com/modelcontextprotocol/servers
- **GitMCP:** gitmcp.io
- **McKay's Template:** github.com/mckaywrigley/mckays-app-template
- **Claude Code CLI:** docs.anthropic.com/en/docs/claude-code
- **Playwright:** playwright.dev

Stack Documentation

- **Next.js:** nextjs.org/docs
- **shadcn/ui:** ui.shadcn.com
- **Supabase:** supabase.com/docs
- **Clerk:** clerk.com/docs
- **Stripe:** stripe.com/docs
- **Drizzle ORM:** orm.drizzle.team
- **Vercel:** vercel.com/docs